Implementing Hybrid Rendering

Introduction

This guide shows you how to configure a Blazor application for hybrid rendering, combining server-side and client-side approaches for optimal performance and flexibility.

Steps to Implement Hybrid Rendering

1. Set Up Your Blazor Application:

- Create a Blazor Web App project using Visual Studio.
- Add necessary dependencies and libraries based on your rendering needs.

2. Configure render modes in Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services

    .AddRazorComponents()     // core services

    .AddInteractiveServerRenderMode()  // SignalR interactivity

    .AddInteractiveWebAssemblyRenderMode(); // WASM interactivity
```

3. Identify Components for Server-Side Rendering:

- Use Blazor Server for components that:
    - Handle sensitive data, such as patient records in healthcare applications.
    - Require frequent server interactions, like real-time financial data.
    - Depend heavily on server resources, such as APIs or secure databases.
- Configure these components to render on the server by adding them to your server project.

Example:

```
@page "/securecomponent"

<SecureComponent />
```

4. Identify Components for Client-Side Rendering:

- Use Blazor WebAssembly for components that:
    - Reduce server load, such as product filtering in an e-commerce site.
    - Work offline, like note-taking features in a travel app.
    - Require interactivity, such as real-time form validation or image editing.
- Add these components to your WebAssembly project and ensure they can function independently of server interactions when required.

Example:

@page "/interactive"

<InteractiveForm @rendermode="InteractiveWebAssembly" /> (<InteractiveForm />)

5. Combine Server and Client-Side Components:

- Mix server-side and client-side rendering in your application based on functionality.
    - Example setup for an educational app:
        - Blazor Server: Secure login and real-time data updates.
        - Blazor WebAssembly: Quizzes and offline lesson plans.

6. Optimize Your Application:

- Ensure server-rendered components minimize latency by optimizing database queries.
- Use caching for client-rendered components to enable offline interactions.
- Test your application to verify smooth transitions between server and client rendering.

Conclusion

By assigning the right **@rendermode** to each component in **a Blazor Web App** , you can deliver secure, high-performance features alongside offline-capable, responsive UI—all without juggling multiple projects.